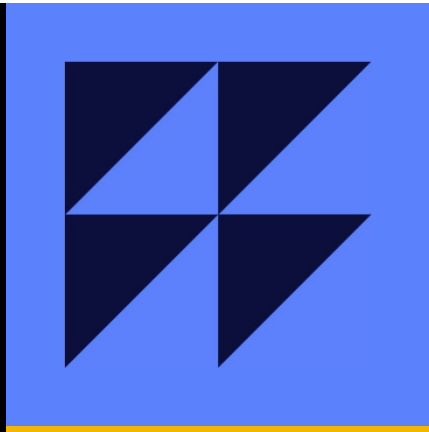
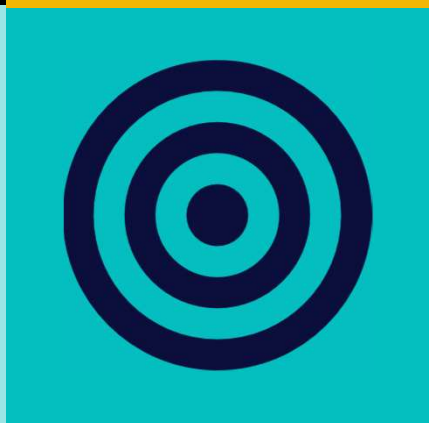
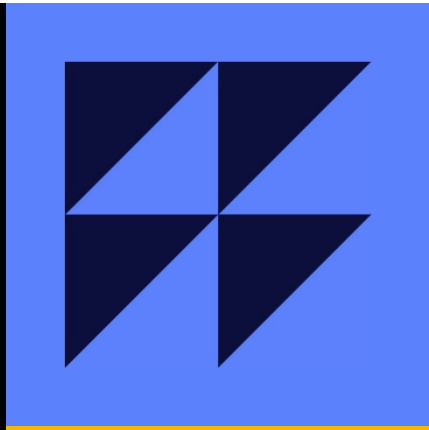
Reinforcement Pre-Training:
RL for Next Token Prediction





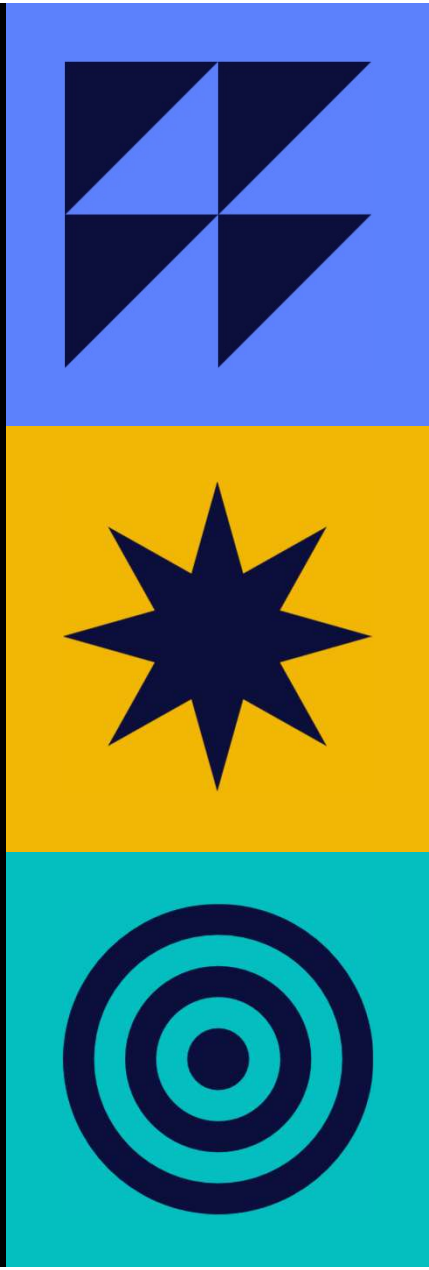
2026.6.24
이태교

Table



Every question is a good question
(there is no stupid question)

Discussion is much funnier
than just speaking and listening



1. What This Paper Is About

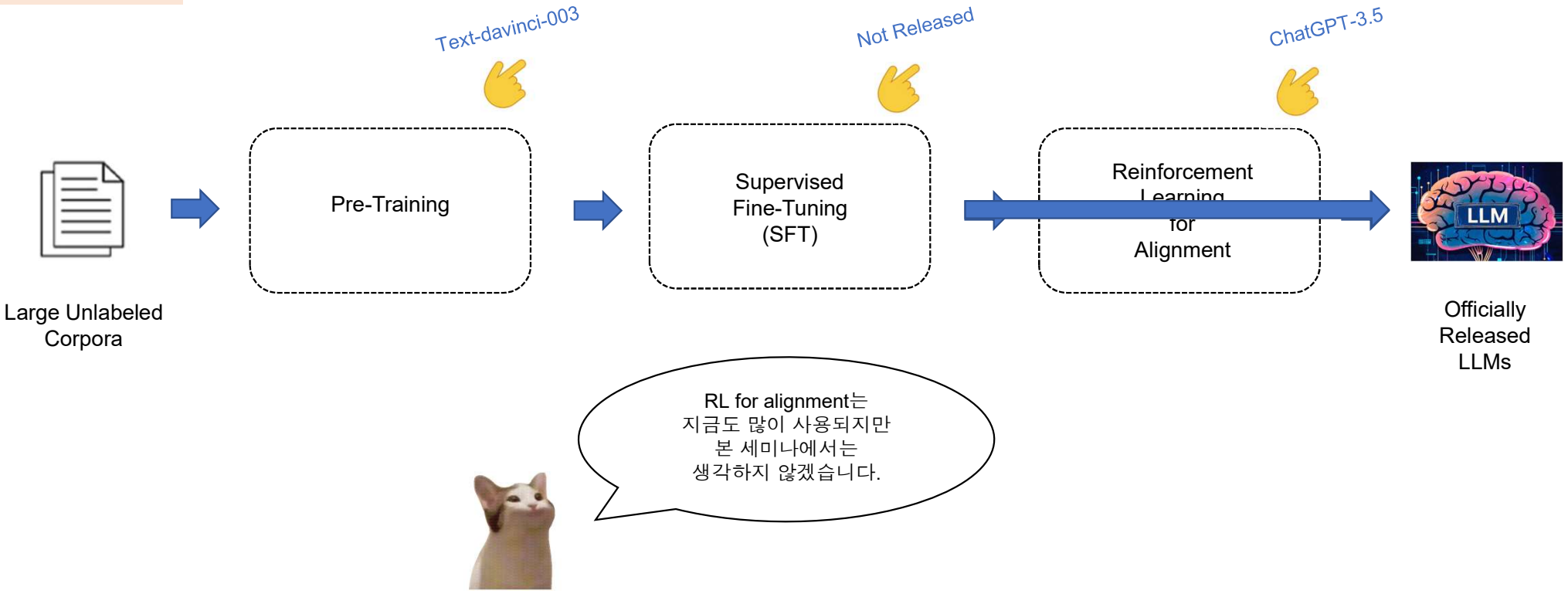
Reinforcement Pre-Training

Qingxiu Dong*†‡ Li Dong*†
Yao Tang† Tianzhu Ye†§ Yutao Sun†§ Zhifang Sui‡ Furu Wei†◇
† Microsoft Research
‡ Peking University
§ Tsinghua University
https://aka.ms/GeneralAI

What This Paper Is About

- Reinforcement Learning Gets to the Origin of LLM Training More and More

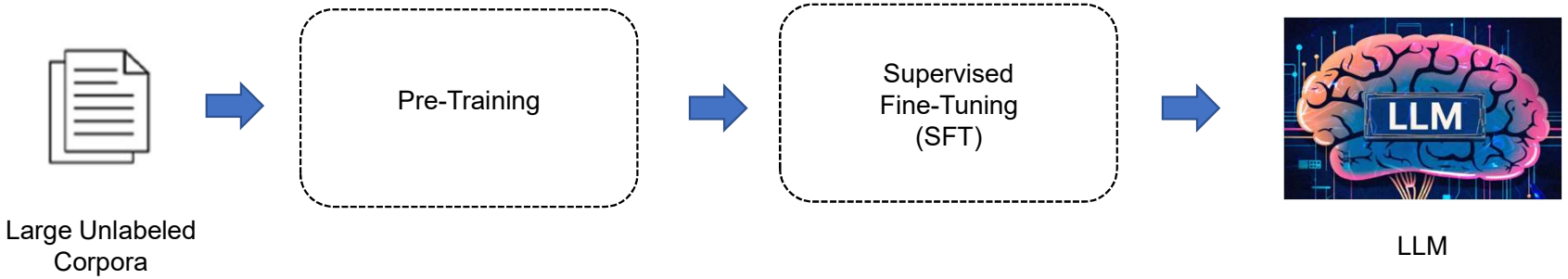
1st paradigm

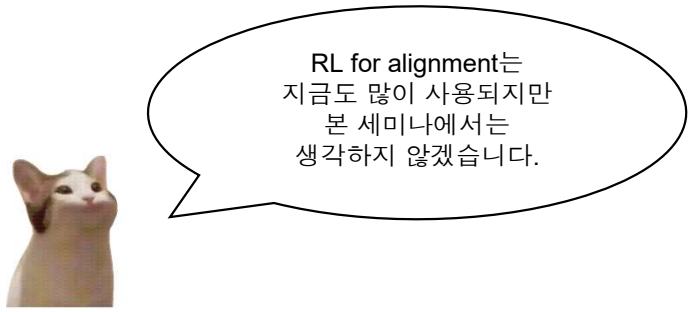


What This Paper Is About

- Reinforcement Learning Gets to the Origin of LLM Training More and More

1st paradigm

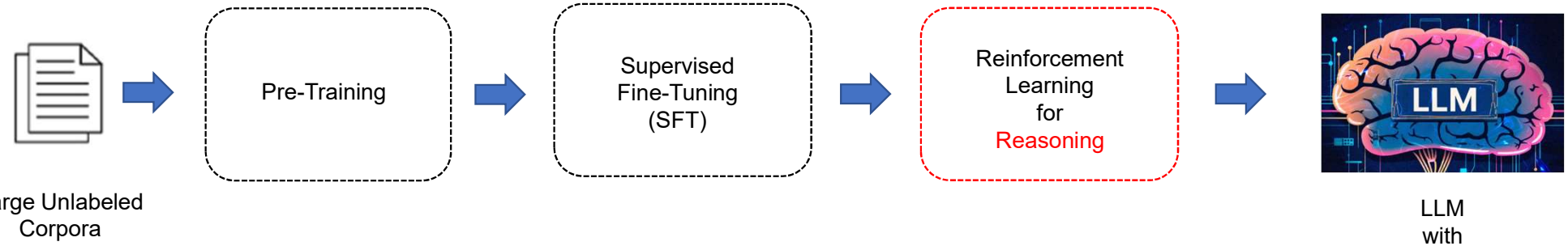




What This Paper Is About

- Reinforcement Learning Gets to the Origin of LLM Training More and More

1.5th paradigm



DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

Apr. 2024

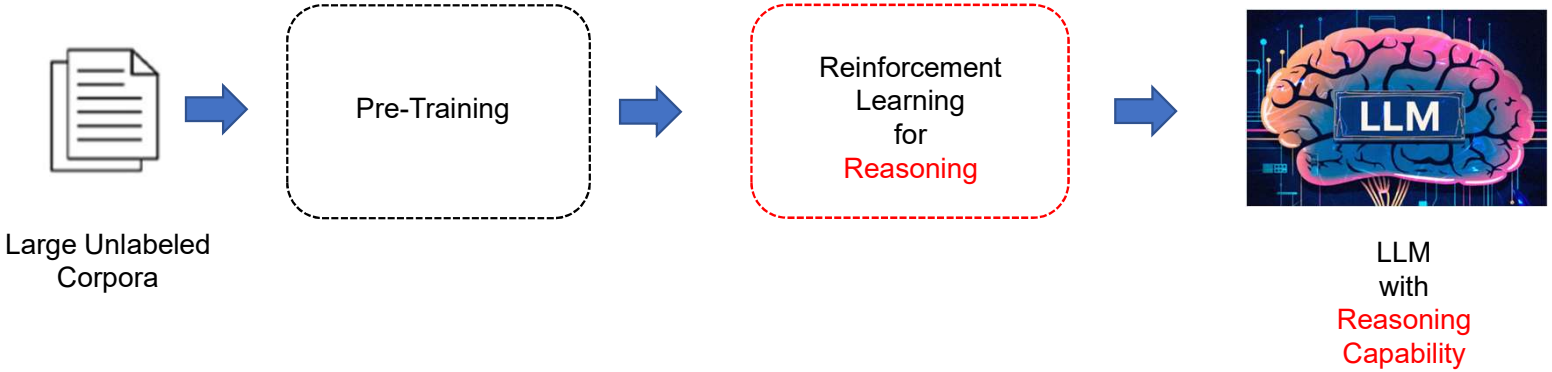
Zhihong Shao[1,2*†], Peiyi Wang[1,3*†], Qihao Zhu[1,3*†], Runxin Xu[1], Junxiao Song[1]
Xiao Bi[1], Haowei Zhang[1], Mingchuan Zhang[1], Y.K. Li[1], Y. Wu[1], Daya Guo[1*]

[1]DeepSeek-AI, [2]Tsinghua University, [3]Peking University

What This Paper Is About

- Reinforcement Learning Gets to the Origin of LLM Training More and More

2nd paradigm



DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

Jan. 2025

DeepSeek-AI

research@deepseek.com

What This Paper Is About

- Reinforcement Learning Gets to the Origin of LLM Training More and More

3rd paradigm?

Large Unlabeled Corpora

Pre-Training

Pre-Training for Reasoning with RL

Optional

Reinforcement Learning for Reasoning

LLM with Reasoning Capability

오늘 소개하려는 논문!

Reinforcement Pre-Training Jun. 2025

Qingxiu Dong*†‡ Li Dong*†
Yao Tang† Tianzhu Ye†§ Yutao Sun†§ Zhifang Sui‡ Furu Wei†◇
† Microsoft Research
‡ Peking University
§ Tsinghua University
https://aka.ms/GeneralAI

What This Paper Is About

- Reinforcement Learning Gets to the Origin of LLM Training More and More

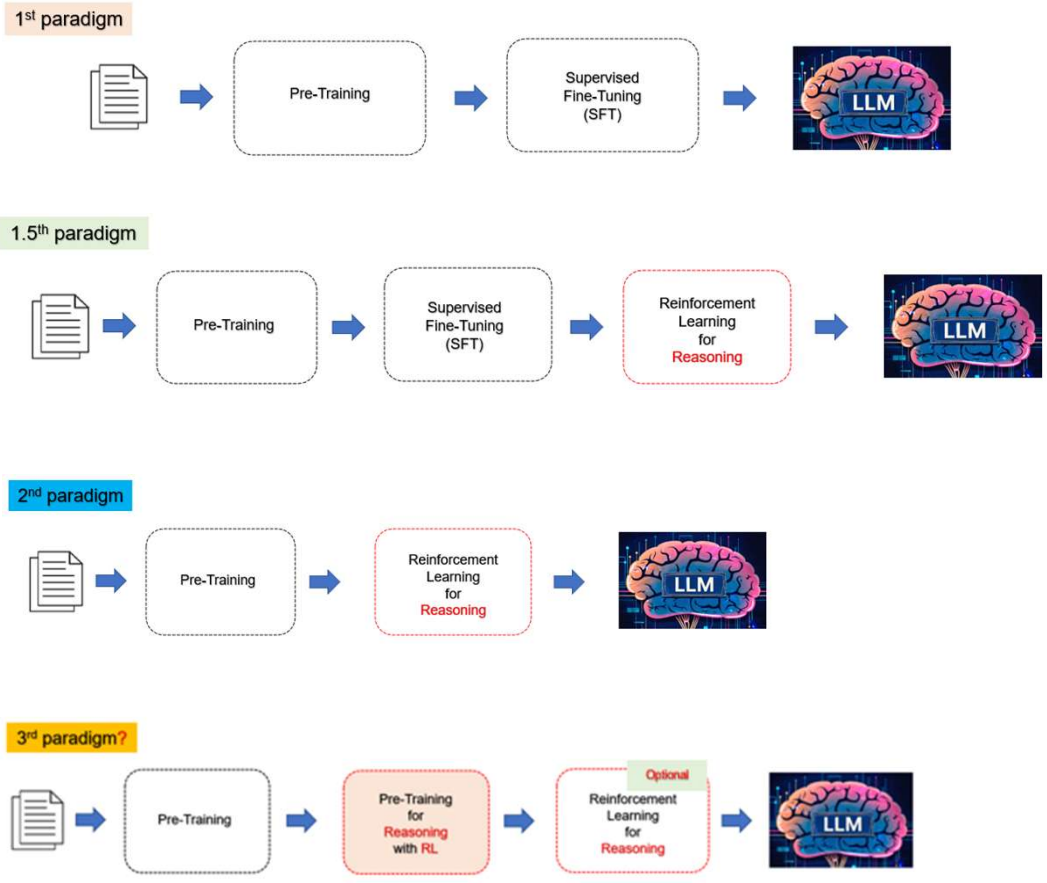
3rd paradigm?

Pre-Training 이라는 용어가
부적절해 보이는데?
누가 들으면 오해하겠어

그 부분에 대해선
Chapter 4, 5에서
다시 얘기합시다.

What This Paper Is About

- Reinforcement Learning Gets to the Origin of LLM Training More and More



RL이 점점
왼쪽으로 이동하고 있네요

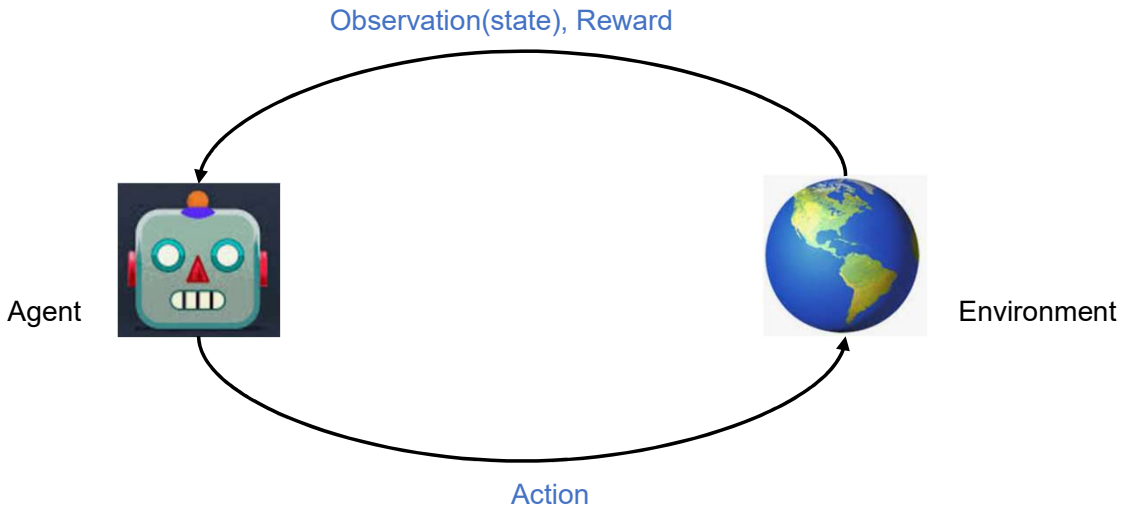
2. Reinforcement Learning to LLMs

RL LLM

Reinforcement Learning to LLMs

Reinforcement Learning is

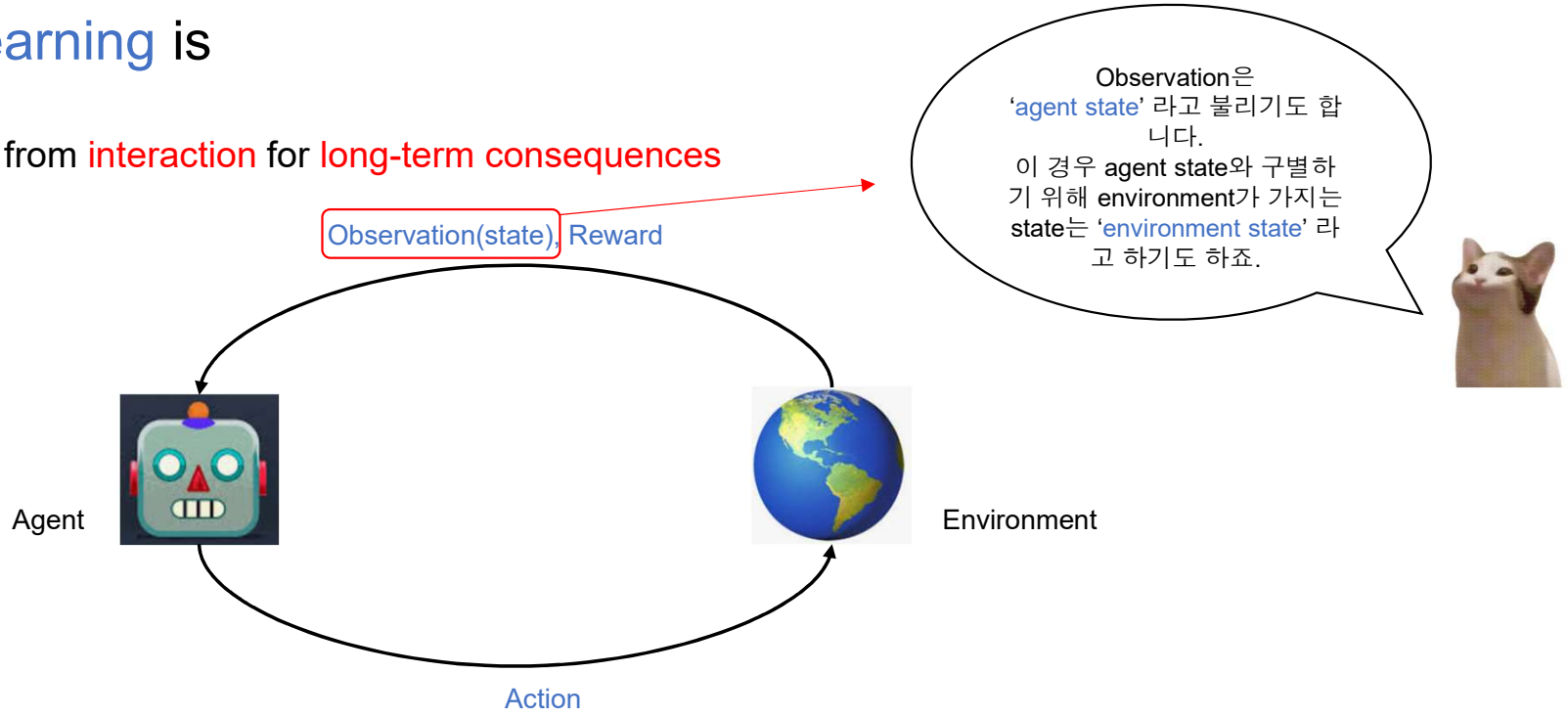
- Learning to make decisions from interaction for long-term consequences



Reinforcement Learning to LLMs

Reinforcement Learning is

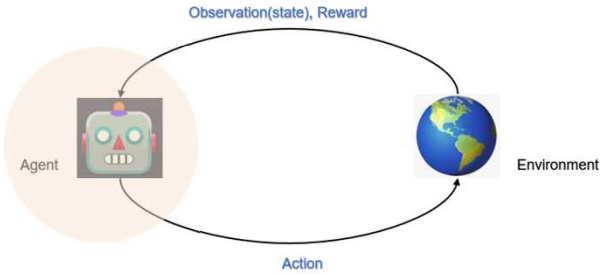
- Learning to make decisions from interaction for long-term consequences



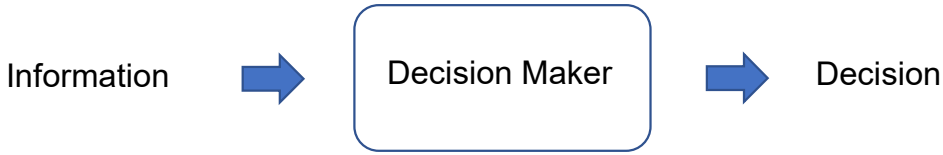
Reinforcement Learning to LLMs

Reinforcement Learning is

- Learning to make decisions from interaction for long-term consequences



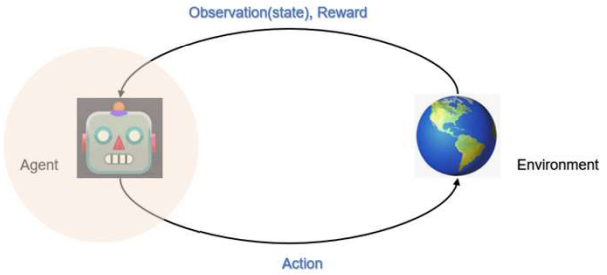
- Agent is a learning system containing
  - ✓ Decision-making mechanism (Policy-based or Value-based or both)
    - **state** (observation or agent state): All ~~necessary~~ information for agent to use in making a decision
    - **action**: Result of a decision
    - **policy**: A mapping from state to action



Reinforcement Learning to LLMs

Reinforcement Learning is

- Learning to make decisions from interaction for long-term consequences



- Agent is a learning system containing
  - ✓ Decision-making mechanism (Policy-based or Value-based or both)
    - **state** (observation or agent state): All ~~necessary~~ information for agent to use in making a decision
    - **action**: Result of a decision
    - **policy**: A mapping from state to action

State s → Policy π_θ → Action a

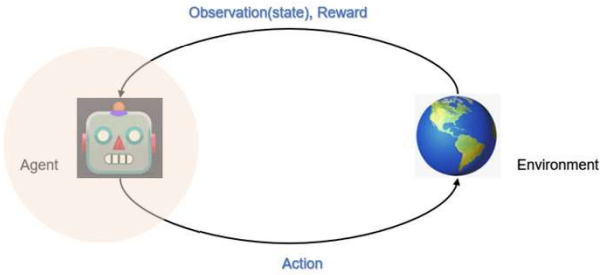
"$\pi_\theta(a|s)$"

Policy를
neural network으로
구성하고
그 policy를 점점
학습시키는 방식을
policy-based RL 이라고
합니다.

Reinforcement Learning to LLMs

Reinforcement Learning is

- Learning to make decisions from interaction for long-term consequences



- Agent is a learning system containing
  - ✓ Decision-making mechanism (Policy-based or Value-based or both)
    - **state** (observation or agent state): All ~~necessary~~ information for agent to use in making a decision
    - **action**: Result of a decision
    - **policy**: A mapping from state to action

State s → Policy π_θ → Action a

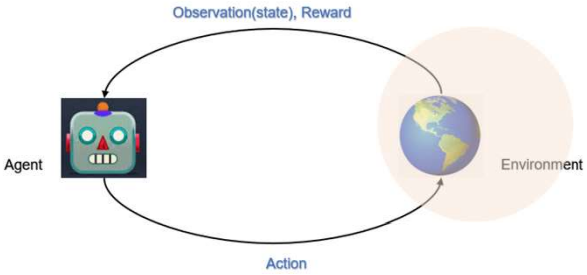
Policy를
neural network으로
구성하고
그 policy를 점점
학습시키는 방식을
policy-based RL 이라고
합니다.

  - ✓ Learning Algorithm (GRPO, Q-learning, A2C, etc.)

Reinforcement Learning to LLMs

Reinforcement Learning is

- Learning to make decisions from interaction for long-term consequences



- Environment is everything around agent
  - ✓ Transition Dynamics: Next-observation(next state)-returning mechanism

Previous state, Previous action

$(s,\ a)$ or $(s_t,\ a_t)$

→ Transition Dynamics

$p(s' \mid s,\ a)$

or

$p(s_{t+1} \mid s_t,\ a_t)$

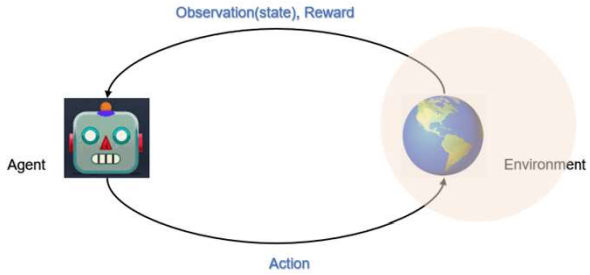
→ Next state

s' or s_{t+1}

Reinforcement Learning to LLMs

Reinforcement Learning is

- Learning to make decisions from interaction for long-term consequences



- Environment is everything around agent
  - ✓ Transition Dynamics: Next-observation(next state)-returning mechanism

Previous state, Previous action

$(s,\ a)$ or $(s_t,\ a_t)$

→ Transition Dynamics

$p(s' \mid s,\ a)$

or

$p(s_{t+1} \mid s_t,\ a_t)$

→ Next state

s' or s_{t+1}

  - ✓ Reward-providing mechanism

- **reward** : scalar signal on 'how well' the agent did at a particular time step

→ model, rule, function, probability distribution, etc.

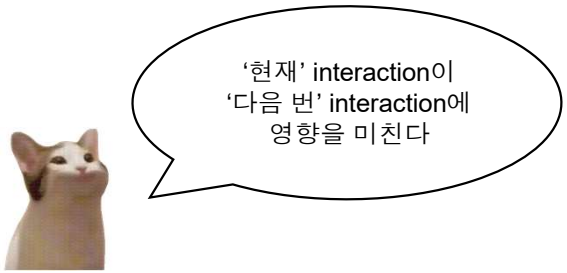
→ Can be delayed or sparse

Reinforcement Learning to LLMs

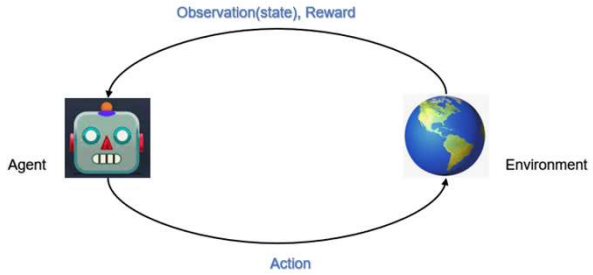
Reinforcement Learning is very different from Supervised Learning

- Learning to make decisions from interaction for long-term consequences

  ✓ Non i.i.d. training data



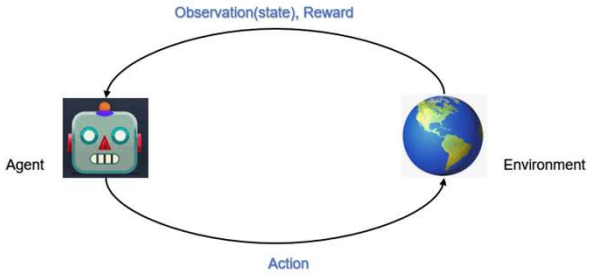
Greedy approach를 못 취함,
Some time-step 에서 sacrifice가 필요함.



Reinforcement Learning to LLMs

Reinforcement Learning is very different from Supervised Learning

- Learning to make decisions from interaction for long-term consequences
  - ✓ Non i.i.d. training data
  - ✓ No Pre-curated data needed



data 없이 학습이 가능하다고???? 부럽다!!!! [1]

SL

미리 만들어 놓은 data를 가져다 쓸 수 있다고???? 부럽다!!!

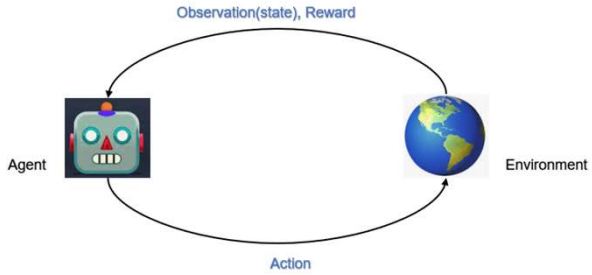
RL

[1] Absolute Zero (https://arxiv.org/pdf/2505.03335)

Reinforcement Learning to LLMs

Reinforcement Learning is very different from Supervised Learning

- Learning to make decisions from interaction for long-term consequences

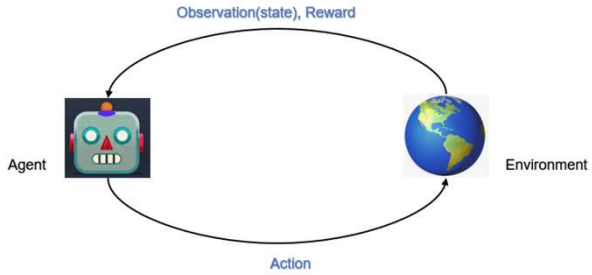


  - ✓ Non i.i.d. training data
  - ✓ No Pre-curated data needed
  - ✓ Learn by trial-and-error Not by examples

Reinforcement Learning to LLMs

Reinforcement Learning is very different from Supervised Learning

- Learning to make decisions from interaction for long-term consequences
  - ✓ Non i.i.d. training data
  - ✓ No Pre-curated data needed
  - ✓ Learn by trial-and-error Not by examples
  - ✓ Maximizing long-term rewards (sum of rewards)



Reward Hypothesis ❗ 중요

In RL, any goal can be formalized as the outcome of maximizing a cumulative reward.

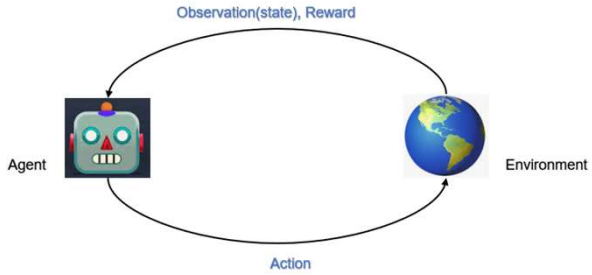
어떠한
Long-term consequence를 optimize 하는 것이
목적이라면,

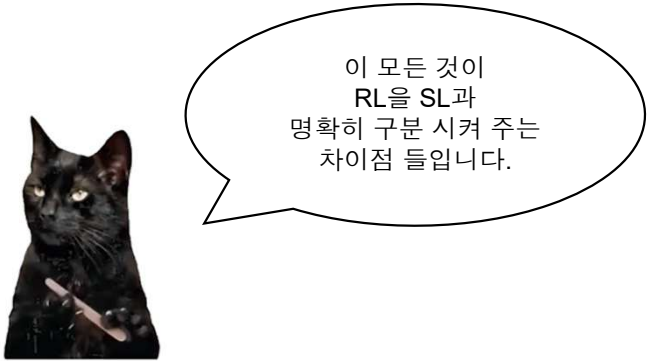
reward의 sum을 maximize 하는 것으로
그 목적을 달성할 수 있는 reward를
언제나 design 할 수 있는가?
라는 말입니다.

Reinforcement Learning to LLMs

Reinforcement Learning is very different from Supervised Learning

- Learning to make decisions from interaction for long-term consequences
  - ✓ Non i.i.d. training data
  - ✓ No Pre-curated data needed
  - ✓ Learn by trial-and-error Not by examples
  - ✓ Maximizing long-term rewards (sum of rewards)





Reinforcement Learning to LLMs

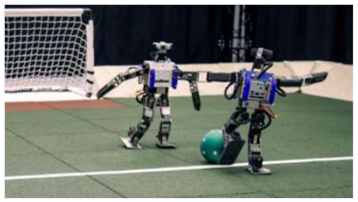
Where Reinforcement Learning comes in?

- Learning to make decisions from interaction for long-term consequences

Reinforcement Learning to LLMs

Where Reinforcement Learning comes in?

- Learning to make decisions from interaction for long-term consequences

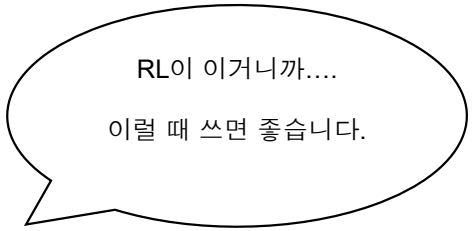


Robotics

Gaming

Autonomous Driving

And



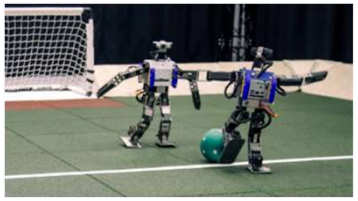
Reinforcement Learning to LLMs

Where Reinforcement Learning comes in?

- Learning to make decisions from interaction for long-term consequences

RL이 이거 니까....

이럴 때 쓰면 좋습니다.



Robotics

Gaming

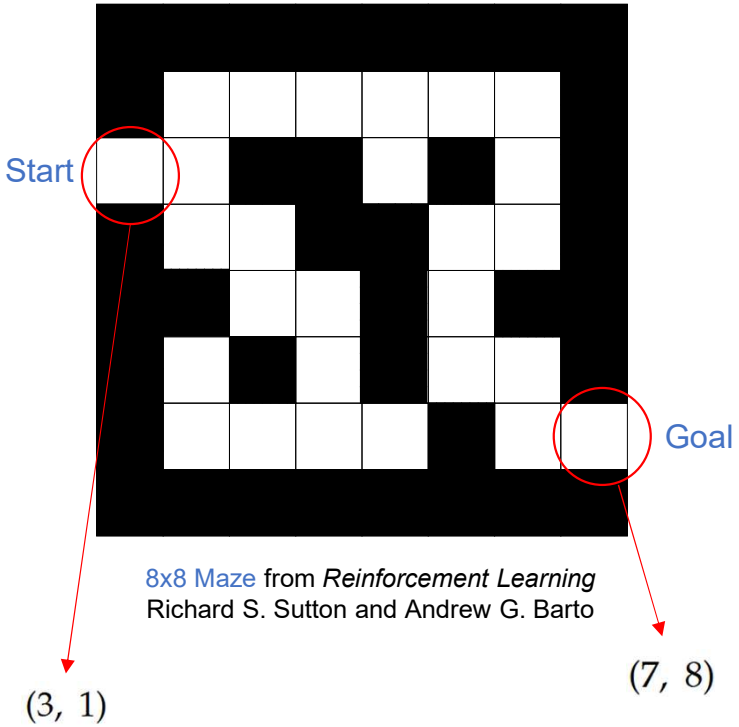
Autonomous Driving

And

LLM

Reinforcement Learning to LLMs

- Why RL and LLM are a good couple?



8x8 Maze from *Reinforcement Learning*
Richard S. Sutton and Andrew G. Barto

State: $(m, n),\ 1 \le m, n \le 8$

Action: $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$

Reward: -1 for each time step

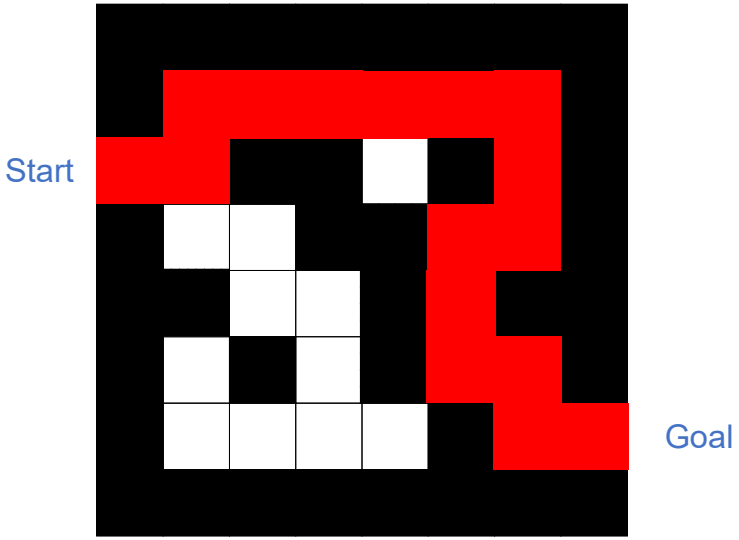
Reward Hypothesis 와
연계하여 생각하면?

Reward Hypothesis

In RL, any goal can be formalized as the outcome of maximizing a cumulative reward.

Reinforcement Learning to LLMs

- Why RL and LLM are a good couple?

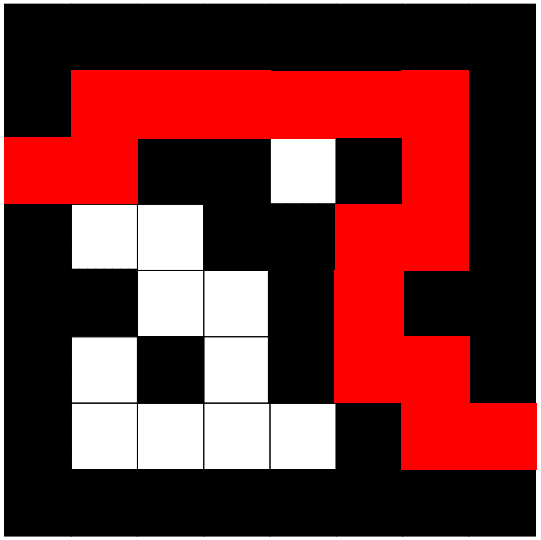


8x8 Maze from Reinforcement Learning
Richard S. Sutton and Andrew G. Barto

Reinforcement Learning to LLMs

- Why RL and LLM are a good couple?

Start



Goal

8x8 Maze from Reinforcement Learning
Richard S. Sutton and Andrew G. Barto

Start

End

Time Step	State	Action	(Cumulative) Reward
$t = 1$	(3, 1)	→	-1
$t = 2$	(3, 2)	↑	-2
$t = 3$	(2, 2)	→	-3
$t = 4$	(2, 3)	→	-4
$t = 5$	(2, 4)	→	-5
$t = 6$	(2, 5)	→	-6
$t = 7$	(2, 6)	→	-7
$t = 8$	(2, 7)	↓	-8
$t = 9$	(3, 7)	↓	-9
$t = 10$	(4, 7)	←	-10
$t = 11$	(4, 6)	↓	-11
$t = 12$	(5, 6)	↓	-12
$t = 13$	(6, 6)	→	-13
$t = 14$	(6, 7)	↓	-14
$t = 15$	(7, 7)	→	-15
$t = 16$	(7, 8)	→	-16

Reinforcement Learning to LLMs

- Why RL and LLM are a good couple?

"안녕" → 

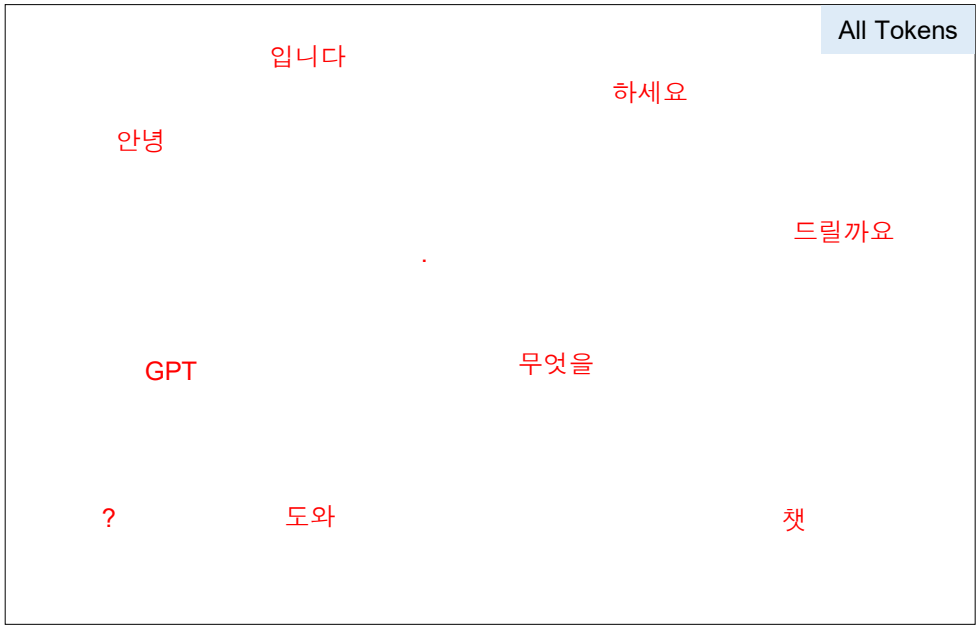
→ "안녕하세요. 챗GPT입니다.
무엇을 도와드릴까요?"

Reinforcement Learning to LLMs

- Why RL and LLM are a good couple?

"안녕" → 

→ "안녕하세요. 챗GPT입니다.
무엇을 도와드릴까요?"

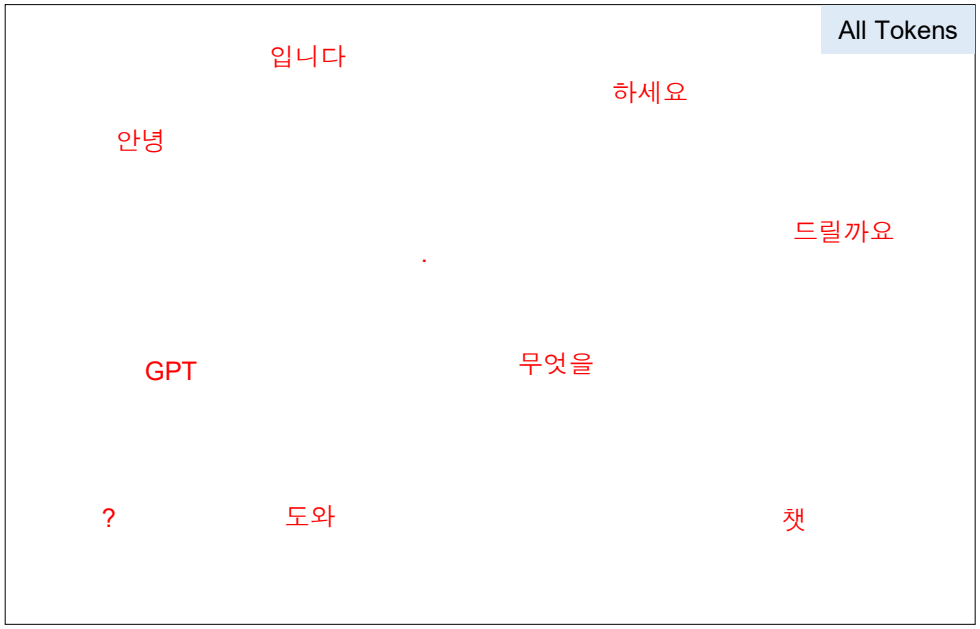


Reinforcement Learning to LLMs

- Why RL and LLM are a good couple?

"안녕" → 

→ "안녕하세요. 챗GPT입니다.
무엇을 도와드릴까요?"



	Time Step	State	Action
Start	$t = 1$		안녕
	$t = 2$	안녕	하세요
	$t = 3$	안녕하세요	.
	$t = 4$	안녕하세요.	챗
	$t = 5$	안녕하세요. 챗	GPT
	$t = 6$	안녕하세요. 챗GPT	입니다
	$t = 7$	안녕하세요. 챗GPT입니다	.
	$t = 8$	안녕하세요. 챗GPT입니다.	무엇을
	$t = 9$	안녕하세요. 챗GPT입니다. 무엇을	도와
	$t = 10$	안녕하세요. 챗GPT입니다. 무엇을 도와	드릴까요
	$t = 11$	안녕하세요. 챗GPT입니다. 무엇을 도와 드릴까요	?
End	$t = 12$	안녕하세요. 챗GPT입니다. 무엇을 도와 드릴까요?	

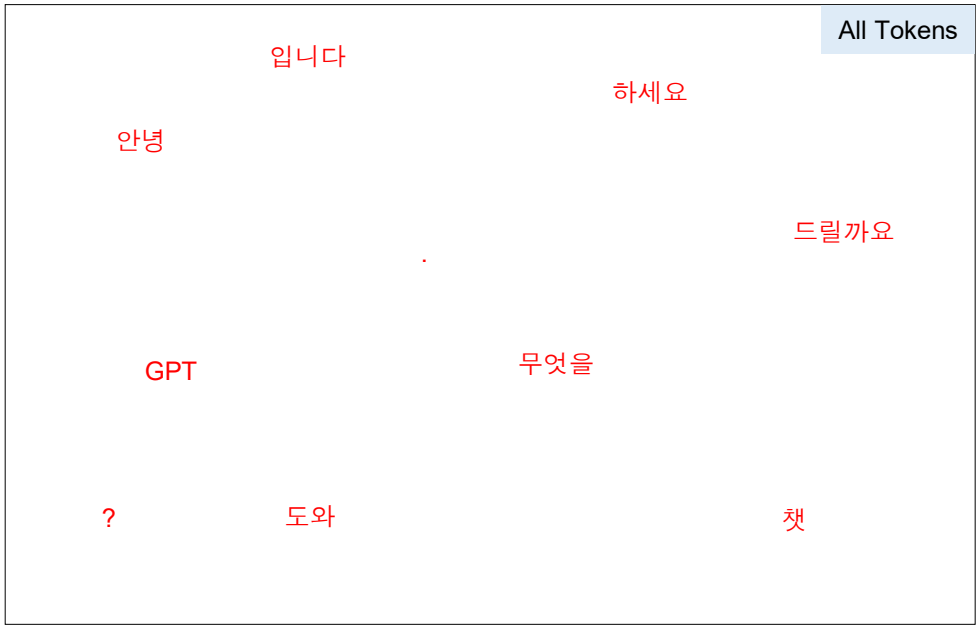
Reinforcement Learning to LLMs

Why RL and LLM are a good couple?

"안녕" → 

→ "안녕하세요. 챗GPT입니다. 무엇을 도와드릴까요?"

RL…
LLM에 적용 안 할 이유가 있나요?



	Time Step	State	Action
Start	$t = 1$		안녕
	$t = 2$	안녕	하세요
	$t = 3$	안녕하세요	.
	$t = 4$	안녕하세요.	챗
	$t = 5$	안녕하세요. 챗	GPT
	$t = 6$	안녕하세요. 챗GPT	입니다
	$t = 7$	안녕하세요. 챗GPT입니다	.
	$t = 8$	안녕하세요. 챗GPT입니다.	무엇을
	$t = 9$	안녕하세요. 챗GPT입니다. 무엇을	도와
	$t = 10$	안녕하세요. 챗GPT입니다. 무엇을 도와	드릴까요
	$t = 11$	안녕하세요. 챗GPT입니다. 무엇을 도와 드릴까요	?
End	$t = 12$	안녕하세요. 챗GPT입니다. 무엇을 도와 드릴까요?	

Reinforcement Learning to LLMs

- Why RL and LLM are a good couple?

Time Step	State	Action	(Cumulative) Reward
$t = 1$	(3, 1)	→	-1
$t = 2$	(3, 2)	↑	-2
$t = 3$	(2, 2)	→	-3
$t = 4$	(2, 3)	→	-4
$t = 5$	(2, 4)	→	-5
$t = 6$	(2, 5)	→	-6
$t = 7$	(2, 6)	→	-7
$t = 8$	(2, 7)	↓	-8
$t = 9$	(3, 7)	↓	-9
$t = 10$	(4, 7)	←	-10
$t = 11$	(4, 6)	↓	-11
$t = 12$	(5, 6)	↓	-12
$t = 13$	(6, 6)	→	-13
$t = 14$	(6, 7)	↓	-14
$t = 15$	(7, 7)	→	-15
$t = 16$	(7, 8)	→	-16

Maze

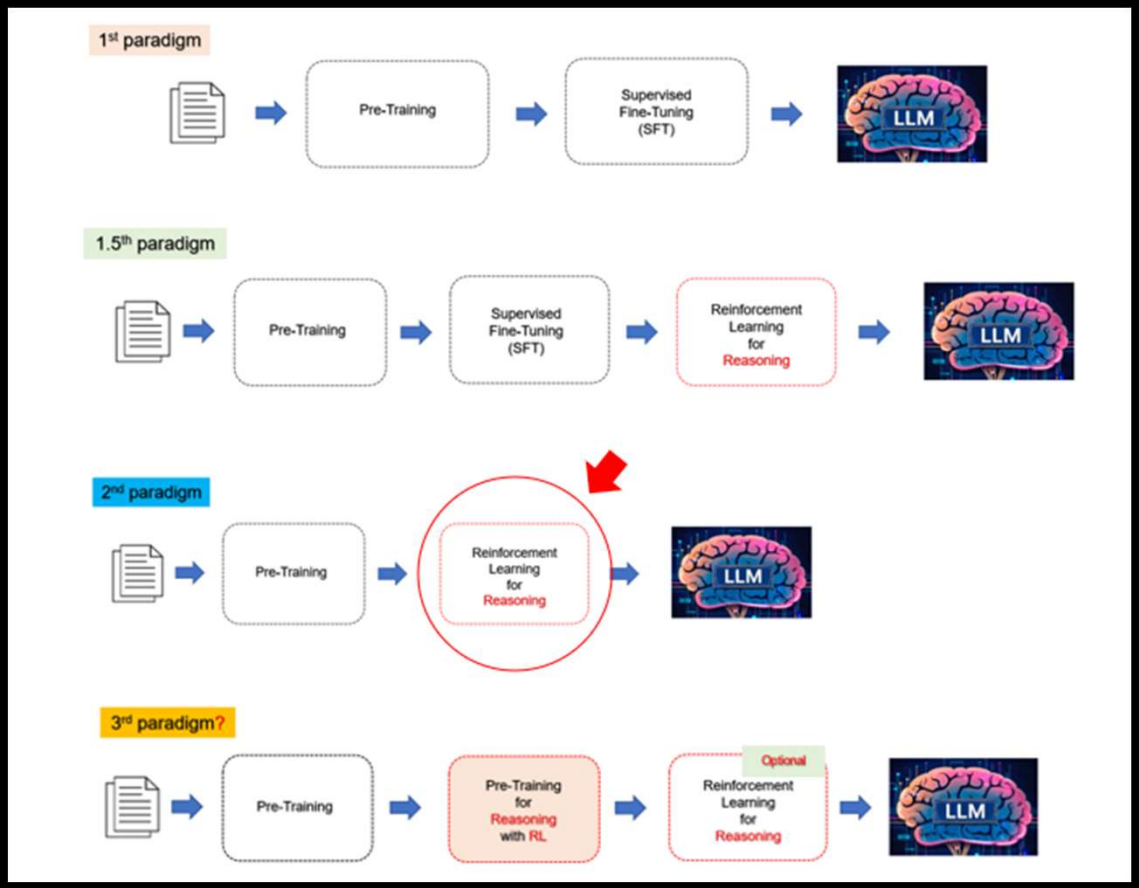
잠깐…
RL for LLM 에서
Reward 어디갔어?

	Time Step	State	Action
Start	$t = 1$		안녕
	$t = 2$	안녕	하세요
	$t = 3$	안녕하세요	.
	$t = 4$	안녕하세요.	챗
	$t = 5$	안녕하세요. 챗	GPT
	$t = 6$	안녕하세요. 챗GPT	입니다
	$t = 7$	안녕하세요. 챗GPT입니다	.
	$t = 8$	안녕하세요. 챗GPT입니다.	무엇을
	$t = 9$	안녕하세요. 챗GPT입니다. 무엇을	도와
	$t = 10$	안녕하세요. 챗GPT입니다. 무엇을 도와	드릴까요
	$t = 11$	안녕하세요. 챗GPT입니다. 무엇을 도와 드릴까요	?
End	$t = 12$	안녕하세요. 챗GPT입니다. 무엇을 도와 드릴까요?	

LLM

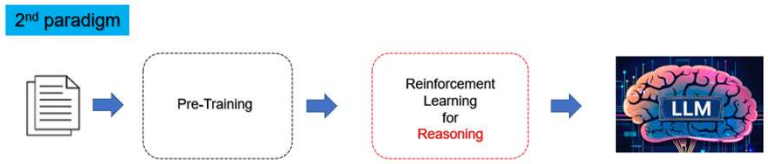
그게 LLM 엔지니어로서
고민해야할 부분이야.
내가 LLM으로 달성하고자
하는 목표가 무엇인가?
그것을 정확히 encoding하는
reward가 무엇인가?

3. Reinforcement Post-Training with Verifiable Reward



Reinforcement Post-Training with Verifiable Reward

Reward for Enhancing Reasoning Capabilities

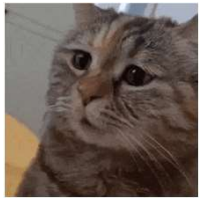


Reward Hypothesis

In RL, any goal can be formalized as the outcome of maximizing a cumulative reward.

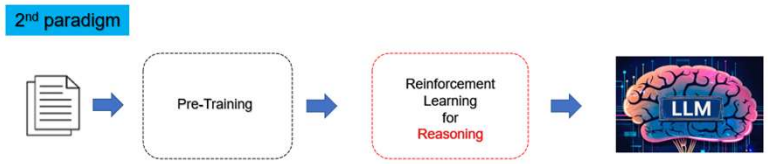
너의 goal은 무엇이냐??

LLM의 reasoning 능력을
향상시키는 것…



Reinforcement Post-Training with Verifiable Reward

Reward for Enhancing Reasoning Capabilities

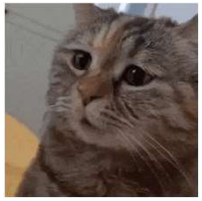


Reward Hypothesis

In RL, any goal can be formalized as the outcome of maximizing a cumulative reward.

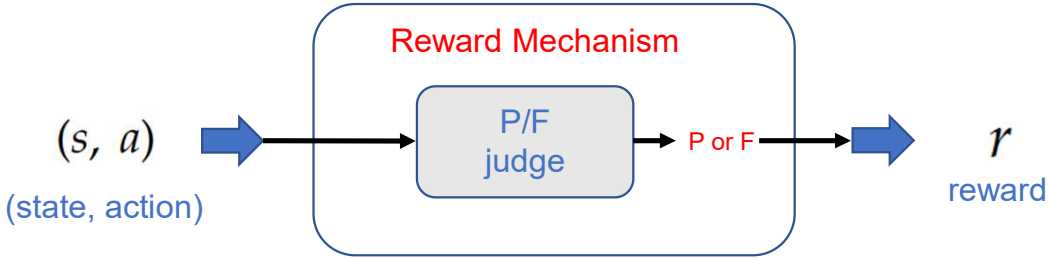
그럼
Summation을 maximize 함으로써
"LLM의 reasoning 능력을
향상시킬 수 있는"
그런 reward 는 무엇이냐?

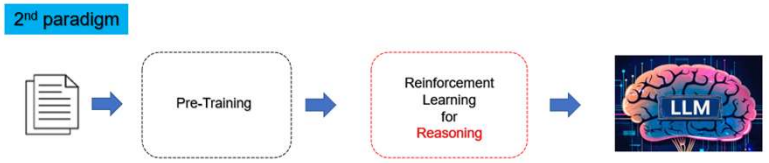
Verifiable Reward….



Reinforcement Post-Training with Verifiable Reward

- Verifiable Rewards





The P/F judgement must be No Doubt!

'의심의 여지가 없는'
P/F 판정을 위해서는
agent가 현재 다루고 있는
task의 종류도 중요합니다.

코딩, 수학, 답변 포맷 등

Reinforcement Post-Training with Verifiable Reward

Verifiable Rewards

- Appropriate Tasks for RL with Verifiable Rewards (RLVR)

Coding

Model Input: Here is the function `f` and the output: '1', please predict the input.

```
def f(numbers: list[int], target: int) -> int:
    accumulated_values = []
    current_sum = 0
    for number in numbers:
        current_sum += number
        accumulated_values.append(current_sum)
    result = 0
    for i in range(len(accumulated_values)):
        for j in range(i + 1, len(accumulated_values)):
            if accumulated_values[j] \
                    - accumulated_values[i] == target:
                result = accumulated_values[i]
                return result
    return result
```

State (s)

Model Output:

Let's analyze the code snippet to understand how the result is achieved. The function f takes a list of integers numbers and a target integer target. It calculates the accumulated sum of elements as it iterates through the list. Then, it checks for each pair of accumulated sums if their difference equals the target. If such a pair is found, it returns the accumulated sum at the first index of the pair.

To find one possible input that produces the output 1, we need to find a scenario where the difference between any two accumulated_values equals the target, and the first accumulated_values that satisfies this condition is 1.
Let's consider a simple example:
-numbers = [1, 2]
-target = 1
Here's the step-by-step calculation:
1. current_sum = 0 + 1 = 1, accumulated_values = [1]
2. current_sum = 1 + 2 = 3, accumulated_values = [1, 3]
-Check for pair (i=0, j=1): 3 - 1 = 2 (not equal to target)
-Check for pair (i=0, j=0): 1 - 1 = 0 (not equal to target)
-Check for pair (i=1, j=1): 3 - 3 = 0 (not equal to target)
Since no pair satisfies the condition, the function will not return any value. This doesn't match our output. Let's try another example.
Consider:
-numbers = [1, 0]
-target = 1
Step-by-step calculation:
1. current_sum = 0 + 1 = 1, accumulated_values = [1]
2. current_sum = 1 + 0 = 1, accumulated_values = [1, 1]
-Check for pair (i=0, j=1): 1 - 1 = 0 (not equal to target)
-Check for pair (i=0, j=0): 1 - 1 = 0 (not equal to target)
-Check for pair (i=1, j=1): 1 - 1 = 0 (not equal to target)
...(shortened)

The pair (i=1, j=2) satisfies the condition, and the function returns the first accumulated value in this pair, which is 3. This still doesn't match our output.

Given the complexity and the fact that no simple example seems to work, let's try a more complex scenario:
-numbers = [2, -1, 1]
-target = 1
Step-by-step calculation:
1. current_sum = 0 + 2 = 2, accumulated_values = [2]
2. current_sum = 2 - 1 = 1, accumulated_values = [2, 1]
3. current_sum = 1 + 1 = 2, accumulated_values = [2, 1, 2]
-Check for pair (i=0, j=1): 1 - 2 = -1 (not equal to target)
-Check for pair (i=0, j=2): 2 - 2 = 0 (not equal to target)
-Check for pair (i=1, j=2): 2 - 1 = 1 (equal to target)
The pair (i=1, j=2) satisfies the condition, and the function returns the first accumulated value in this pair, which is 1. This matches our output.

Therefore, one possible input that could produce the output 1 is: [2, -1, 1], 1

Final action (a)

Reinforcement Post-Training with Verifiable Reward

Verifiable Rewards

- Appropriate Tasks for RL with Verifiable Rewards (RLVR)

Math

```
Let $a,$ $b,$ $c$ be the roots of the cubic $x^3 + 3x^2 + 5x + 7 = 0.$ Given that $P(x)$ is a
cubic polynomial such that $P(a) = b + c,$ $P(b) = a + c,$ $P(c) = a + b,$ and $P(a + b + c)
= -16,$ find $P(x).$
```

State (s)

```
Let $Q(x) = P(x) - x - 2.$ Then
\begin{align*}
Q(a) &= b + c - a - 2 = 0, \\
Q(b) &= a + c - b - 2 = 0, \\
Q(c) &= a + b - c - 2 = 0, \\
Q(a + b + c) &= -16 - (a + b + c) - 2 = -18 - (a + b + c).
\end{align*}Since $a,$ $b,$ $c$ are the roots of $x^3 + 3x^2 + 5x + 7 = 0,$
\[(x - a)(x - b)(x - c) = x^3 + 3x^2 + 5x + 7.\]We can then compute that
\begin{align*}
Q(a + b + c) &= (-18 - (a + b + c)) \\
&= -(a + b + c) - 18 \\
&= -(a + b + c) - (a + b + c) - 7 \\
&= -3(a + b + c) - 7 \\
&= -(a + b + c)(a + b + c + 3) \\
&= -(a + b + c)[(a - a) + (b - b) + (c - c) + 3] \\
&= -3(a + b + c).
\end{align*}Since $Q(a + b + c) = -18 - (a + b + c),$ we have $3(a + b + c) = 18.$ But $a + b
+ c$ is the negation of the coefficient of $x^2$ in $x^3 + 3x^2 + 5x + 7,$ which is $-3,$ so
$18 = 3(a + b + c) = 9.$ Hence, $Q(x) = -18 - x.$

Thus, $P(x) = Q(x) + x + 2 = \boxed{-16 - x}.$
```

Final action (a)

Reinforcement Post-Training with Verifiable Reward

Verifiable Rewards

- Appropriate Tasks for RL with Verifiable Rewards (RLVR)

Format

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <think> </think> and <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>. User: prompt. Assistant:

State (s)

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a+x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a+x}} = x$, let's start by squaring both $\cdots$

$\left(\sqrt{a - \sqrt{a+x}}\right)^2 = x^2 \implies a - \sqrt{a+x} = x^2.$

Rearrange to isolate the inner square root term:

$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$

$\ldots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

$\sqrt{a - \sqrt{a+x}} = x$

First, let's square both sides:

$a - \sqrt{a+x} = x^2 \implies \sqrt{a+x} = a - x^2$

Next, I could square both sides again, treating the equation: $\cdots$

$\ldots$

Final action (a)

Reinforcement Post-Training with Verifiable Reward

Verifiable Rewards

- Inappropriate Tasks for RL with Verifiable Rewards (RLVR)

State (s)

강화학습에 대해서 500자 내외로 알기 쉽게 설명해줘

Final action (a)

강화학습(Reinforcement Learning, RL)은 ***시행착오를 통해 최적의 행동을 학습하는 인공지능 방법론***입니다. 애플리케이션의 목표는 **보상(Reward)**을 최대화하는 전략을 스스로 찾는 것이며, 인간이 학습하는 방식과 유사합니다.

핵심 개념

1. **에이전트(Agent)**: 학습 주체 (예: 게임 캐릭터, 자율주행차).
2. **환경(Environment)**: 에이전트가 상호작용하는 공간 (예: 게임 맵, 도로).
3. **상태(State)**: 환경의 현재 상황 (예: 현재 위치, 점수).
4. **행동(Action)**: 에이전트의 선택 (예: 이동, 점프).
5. **보상(Reward)**: 행동의 결과로 얻는 점수 (예: 점수 증가, 패널티).

작동 원리

- 에이전트는 **탐험(Exploration)**과 **활용(Exploitation)** 사이에서 균형을 잡습니다.
  → 새로운 행동을 시도하거나(탐험), 기존의 최적 행동을 반복(활용)합니다.
- **Q-Learning, Deep Q-Network(DQN)** 같은 알고리즘으로 미래 보상의 총합을 예측하며 학습합니다.

예시

- **알파고**: 바둑에서 최적의 수를 탐색하며 승리합니다.
- **로봇 제어**: 넘어지지 않고 걷는 방법을 터득합니다.
- **추천 시스템**: 사용자 반응을 보상으로 삼아 개인화된 추천을 제공합니다.

특징

- **지도학습과 차이점**: 정답 레이블이 없고, 보상이 지연될 수 있습니다.
- **도전 과제**: 복잡한 환경에서는 학습 속도가 느리거나 불안정할 수 있습니다.

강화학습은 **자율성**과 **적응력**이 필요한 분야에서 혁신을 이끌며, 머신러닝의 핵심 기술로 자리잡고 있습니다.

→ P/F judge →

Reinforcement Post-Training with Verifiable Reward

Verifiable Rewards

- Inappropriate Tasks for RL with Verifiable Rewards (RLVR)

State (s)

f(x) = x^5 + 3x^2 + 2 의 이차 도함수를 구해줘

Final action (a)

Let me calculate the second order derivative of f(x) = x^5 + 3x^2 + 2. Check if i am correct. Step 1: Compute the first derivative (f'(x))
We differentiate each term with respect to (x):

f'(x) = \frac{d}{dx}(x^5) + \frac{d}{dx}(3x^2) + \frac{d}{dx}(2)

f'(x) = 5x^4 + 6x + 0 \quad (\text{since the derivative of a constant is } 0)

f'(x) = 5x^4 + 6x

Step 2: Compute the second derivative (f''(x))
Now, differentiate (f'(x)) with respect to (x):

f''(x) = \frac{d}{dx}(5x^4) + \frac{d}{dx}(6x)

f''(x) = 20x^3 + 6

Thus f''(x) = 20x^3 + 6

→ LLM as P/F judge →

Your calculation of the second derivative is correct. Here's a step-by-step verification:

Given:

[f(x) = x^5 + 3x^2 + 2]

Step 1: Compute the first derivative (f'(x))

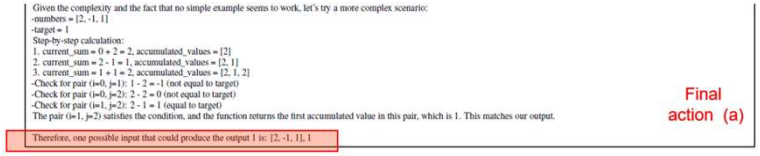
Differentiate each term with respect to (x):

[
\frac{d}{dx}(x^5) = 5x^4
]
[

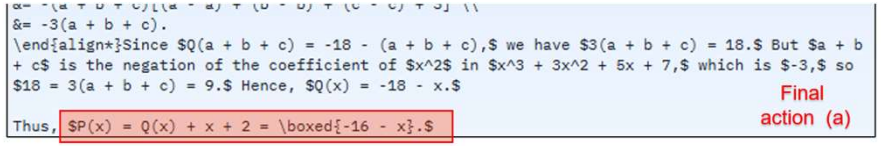
Reinforcement Post-Training with Verifiable Reward

Short Discussion

- Why ‘Final’ action?



- Labeled dataset for RLVR?

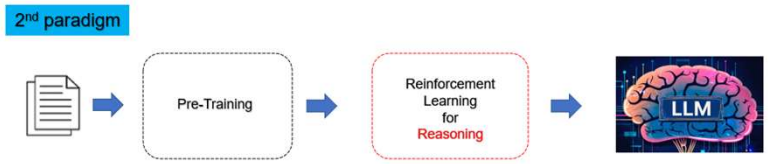


- Can reasoning capabilities cross over to different domain?

- May RLVR harm non-reasoning tasks?

Reinforcement Post-Training with Verifiable Reward

- Examples of Post-Training with RLVR



DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

DeepSeek R1

OpenAI o1

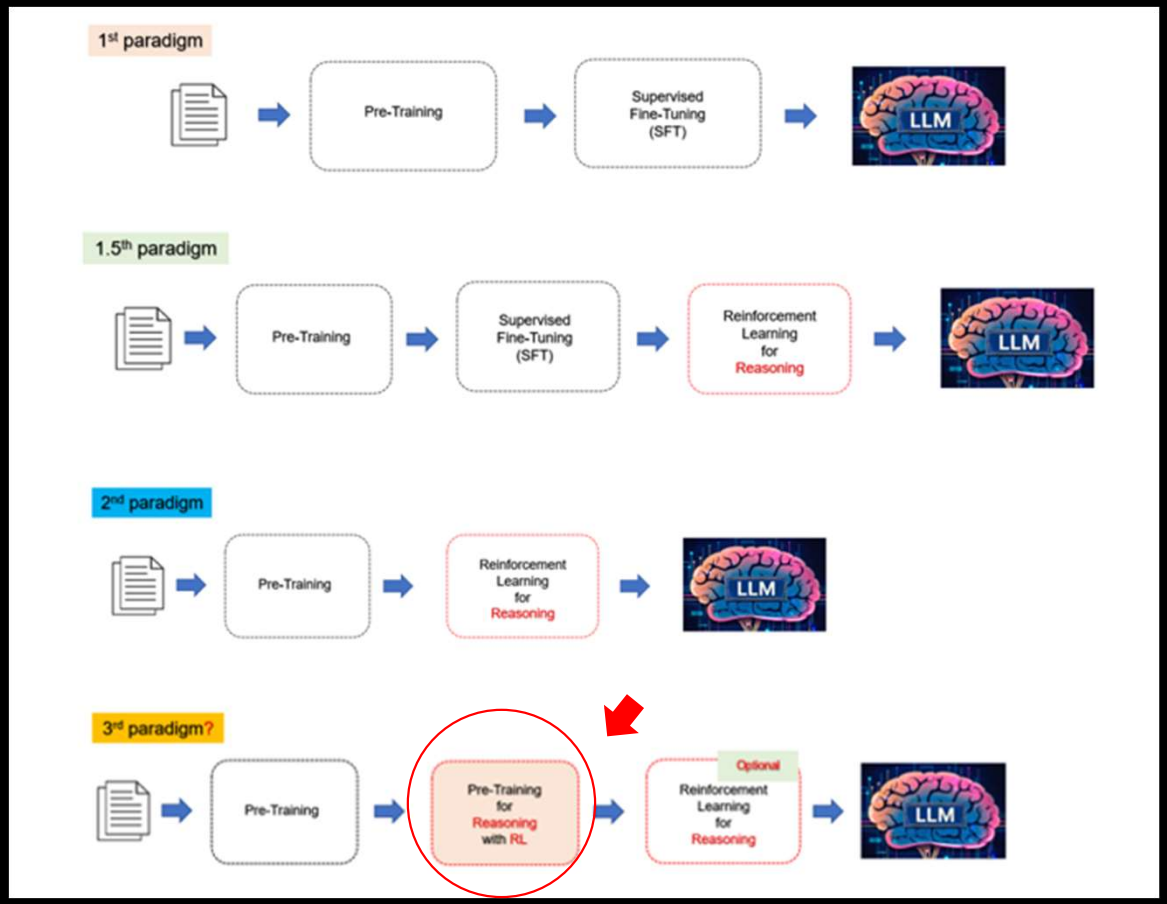
Absolute Zero: Reinforced Self-play Reasoning with Zero Data

Andrew Zhao[1], Yiran Wu[3], Yang Yue[1], Tong Wu[2], Quentin Xu[1], Yang Yue[1], Matthieu Lin[1], Shenzhi Wang[1], Qingyun Wu[3], Zilong Zheng[2,✉] and Gao Huang[1,✉]

[1] Tsinghua University [2] Beijing Institute for General Artificial Intelligence [3] Pennsylvania State University

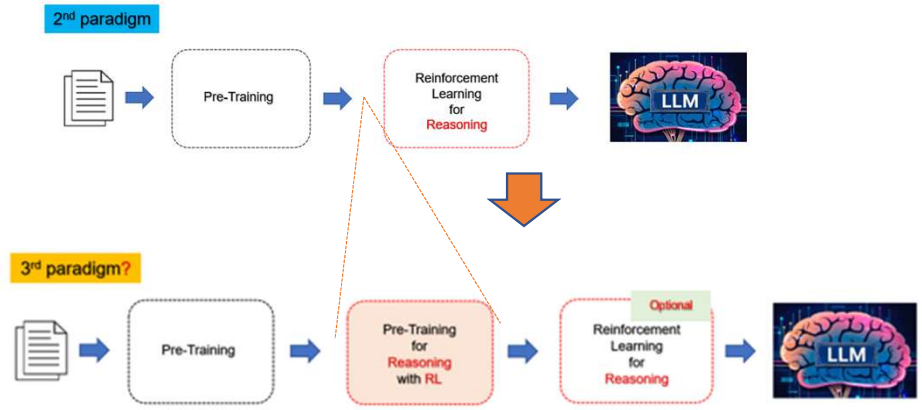
100% online RL

4. New Paradigm? Reinforcement Pre-Training



New Paradigm? Reinforcement Pre-Training

What? : Reasoning for Next-Token Prediction

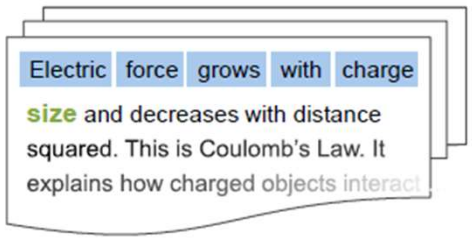


- Reinforcement Pre-Training (RPT) Predicts the Next Token through Reasoning (<think> ~~~~ </think>)

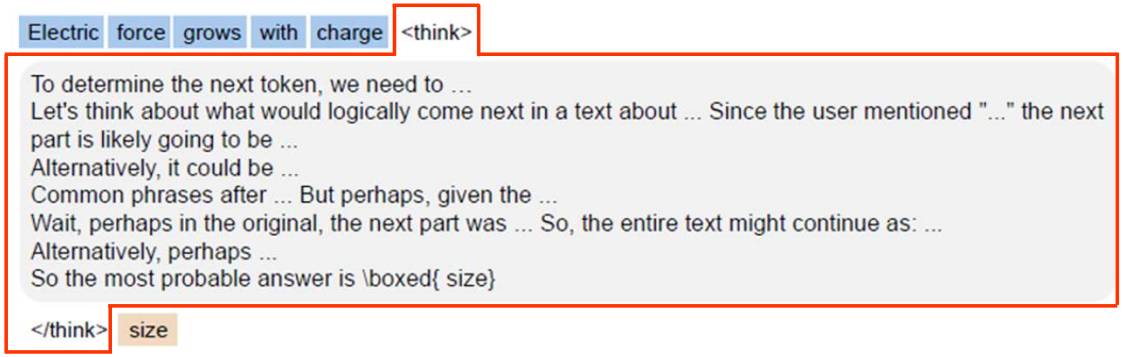
기존의 Pre-Training stage를
대체 하는게 아니네..

New Paradigm? Reinforcement Pre-Training

- **How?** : Reasoning for Next-Token Prediction

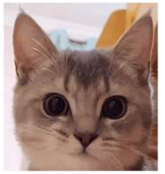


Corpus



너무 예상대론 데...

그 생각을 처음부터
했어야지.
그리고 평범한 아이디어를
바로 실행할 수 있는
추진력과 인프라!!



New Paradigm? Reinforcement Pre-Training

Short Discussion

1) Does it make sense to call it ‘Pre-Training’?

- It CANNOT replace the standard pre-training stage (standard next-token prediction) since the early reasoning capability is only picked up through it.

- Authors claim it is ‘Pre-Training’ because

  - It does not put a domain limitation in training corpus just like normal pre-training

    Coding, Math

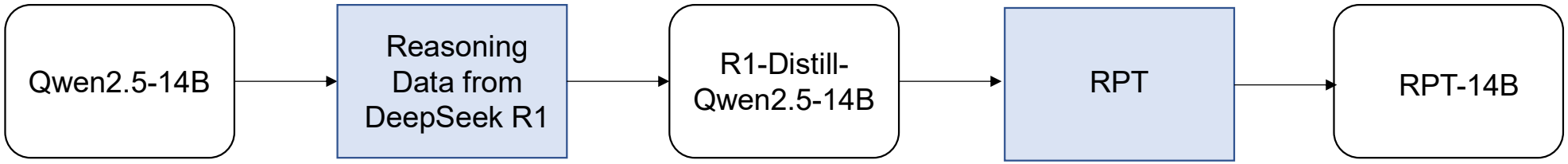
  - After all, it is done by self-supervised fashion.

    Shifting one-token and predict next tokens

context in a pre-training corpus, the model is incentivized to reason about the subsequent token before predicting it. It receives a verifiable, intrinsic reward based on the correctness of its prediction against the ground-truth next token from the corpus itself. This approach transforms the vast, unannotated

New Paradigm? Reinforcement Pre-Training

How Good Is It? : Reasoning for Next-Token Prediction



Token 별 난이도 (R1-Distill-Qwen-14b entropy 기반)

	Easy	Medium	Hard
Standard next-token prediction			
Qwen2.5-14B	41.90	30.03	20.65
R1-Distill-Qwen-14B	41.60	29.46	20.43
Next-token reasoning			
R1-Distill-Qwen-14B	3.31	1.66	1.41
RPT-14B	**45.11**	**33.56**	**23.75**

Table 1: Next-token prediction accuracy across three test splits of varying difficulty. RPT outperforms both the standard next-token prediction baselines and the reasoning-based prediction baseline.

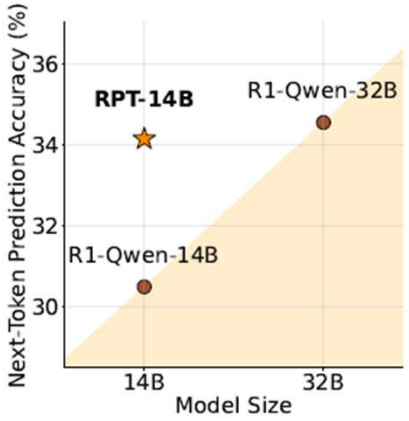
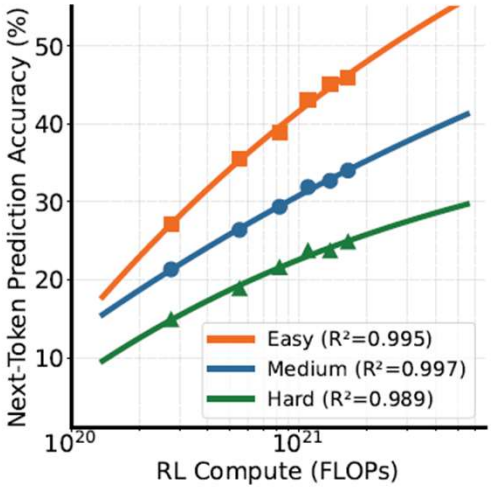
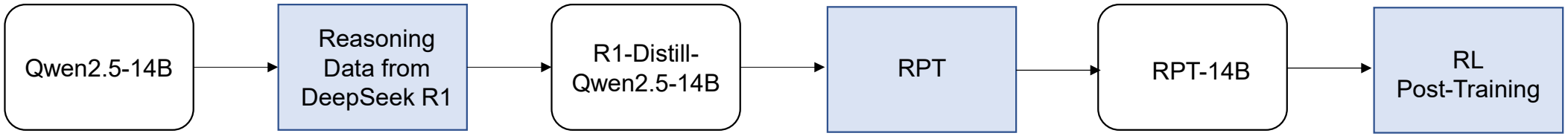


Figure 4: Average next-token prediction accuracy across data of different difficulty levels. R1-Qwen-14B/32B denote R1-Distill-Qwen-14B/32B, respectively.



New Paradigm? Reinforcement Pre-Training

How Good Is It? : Reasoning for Next-Token Prediction



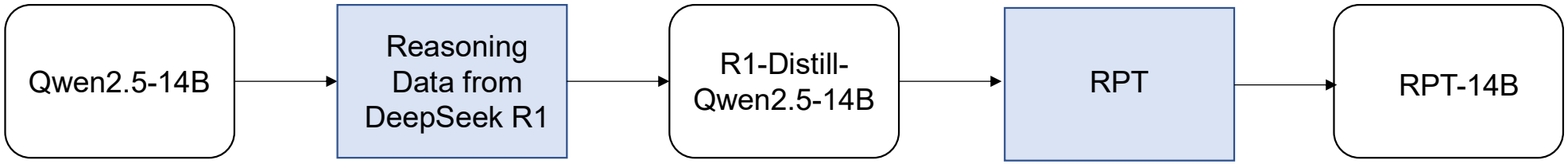
	Before RL	After RL
R1-Distill-Qwen-14B	51.2	52.7
+ Continual NTP training	10.7	13.0
RPT-14B	**56.3** →	**58.3**

추가로 RLVR 로 post-training을 하면
추가로 성능향상을 얻을 수 있다....

추가적인 성능향상이
RPT와 관계 있다는 근거는?

New Paradigm? Reinforcement Pre-Training

How Good Is It? : Reasoning for Next-Token Prediction



Token 별 난이도 (R1-Distill-Qwen-14b entropy 기반)

	Easy	Medium	Hard
Standard next-token prediction			
Qwen2.5-14B	41.90	30.03	20.65
R1-Distill-Qwen-14B	41.60	29.46	20.43
Next-token reasoning			
R1-Distill-Qwen-14B	3.31	1.66	1.41
RPT-14B	**45.11**	**33.56**	**23.75**

Table 1: Next-token prediction accuracy across three test splits of varying difficulty. RPT outperforms both the standard next-token prediction baselines and the reasoning-based prediction baseline.

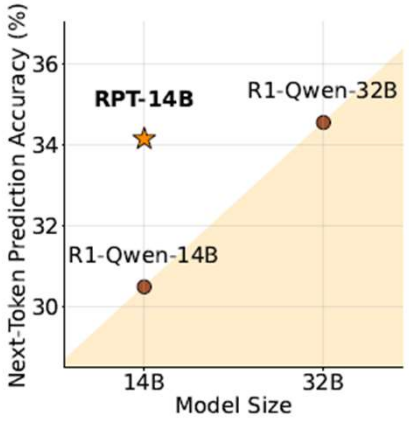
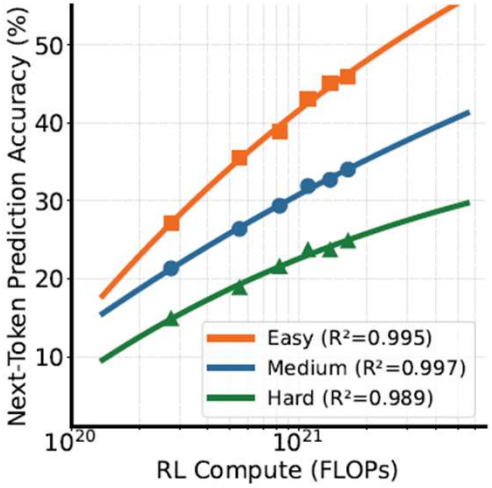


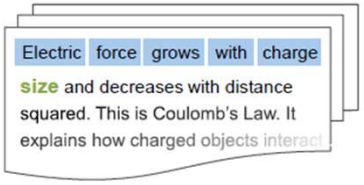
Figure 4: Average next-token prediction accuracy across data of different difficulty levels. R1-Qwen-14B/32B denote R1-Distill-Qwen-14B/32B, respectively.

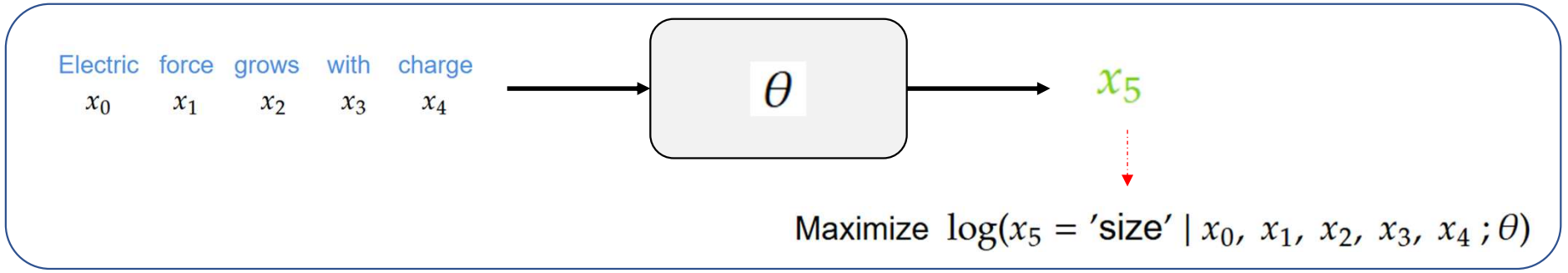


New Paradigm? Reinforcement Pre-Training

- Technical Details

Next-Token Prediction
(Pre-Training of LLM)





$$\mathcal{J}_{\text{NTP}}(\theta) = \sum_{t=1}^{T} \log P(x_t \mid x_0, x_1, \ldots, x_{t-1}; \theta),$$

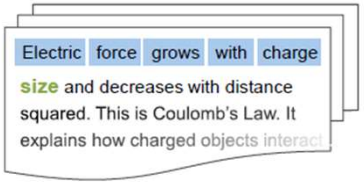
Maximum Likelihood Optimization

Cross-entropy loss
minimization 과
수학적으로 완전히 동일합
니다.

New Paradigm? Reinforcement Pre-Training

- Technical Details

RLVR for LLM



Electric force grows with charge
x_0 x_1 x_2 x_3 x_4

θ → x_5, $r_5 = R(x_5)$

Must be Verifiable

Electric force grows with charge size
x_0 x_1 x_2 x_3 x_4 x_5

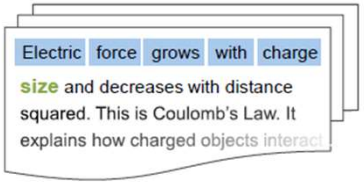
θ → x_6, $r_6 = R(x_6)$

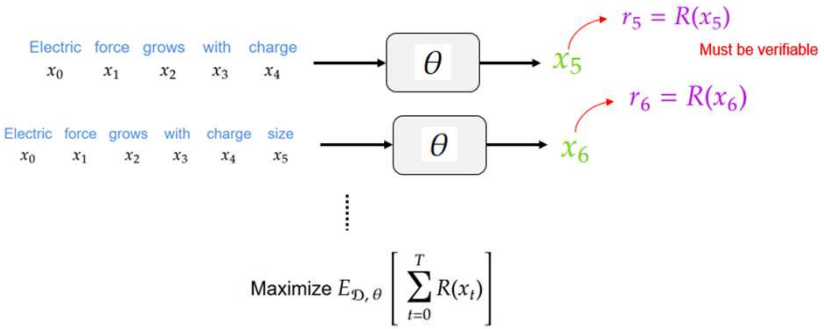
$$\text{Maximize } E_{\mathcal{D},\theta}\left[\sum_{t=0}^{T} R(x_t)\right]$$

New Paradigm? Reinforcement Pre-Training

Technical Details

RLVR for LLM



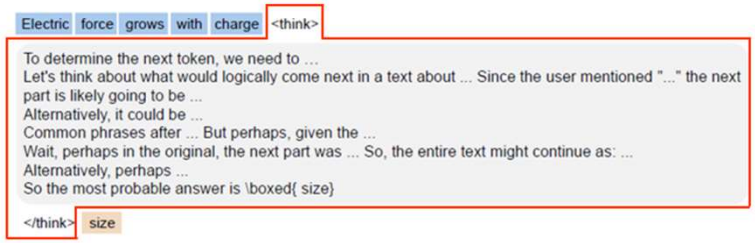


- In practice in RLVR for LLM, LLM is given a reward only when the completion ends. Therefore,

$$\mathcal{J}_{RLVR} = E_{\mathcal{D},\theta}\left[R(x_T)\right],\ x_T=\text{</end>}$$

- In RPT, it is slightly changed

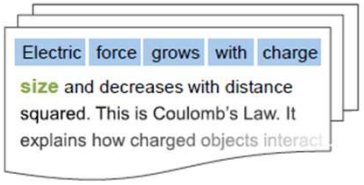
$$\mathcal{J}_{RLVR} = E_{\mathcal{D},\theta}\left[R(x_T)\right],\ x_T=\text{token after </think>}$$

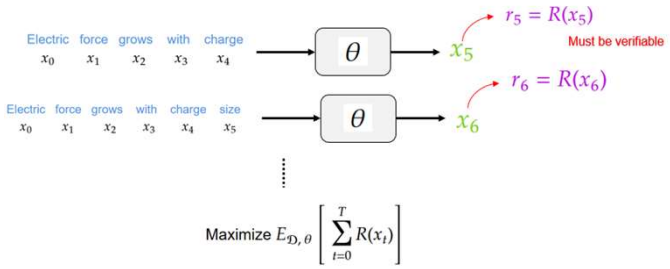


New Paradigm? Reinforcement Pre-Training

Technical Details

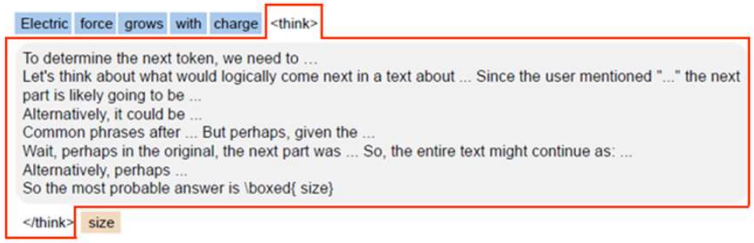
RLVR for LLM





- In RPT, it is slightly changed

$$\mathcal{J}_{RLVR} = E_{\mathcal{D},\theta}\left[R(x_T)\right],\ x_T = \text{token after } \langle/\text{think}\rangle$$



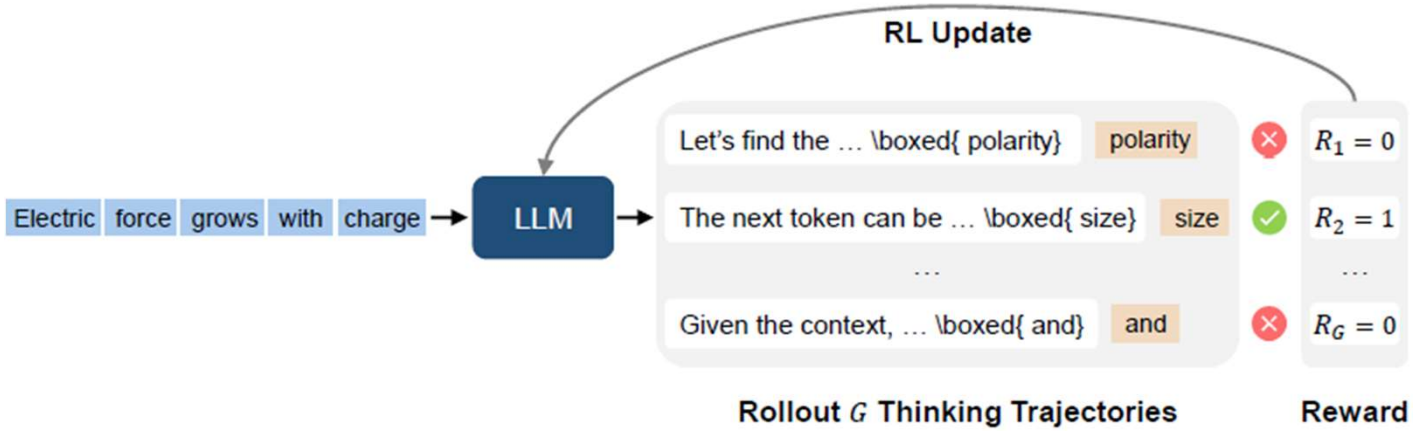
이 논문에 대한 detail은
이제 $R(x_T)$ 를 어떻게
design 했는지만 알면 끝!!

그냥 next token 맞추면
+ reward 주고
틀리면 안 주고 그러면
되는거 아니예요?

New Paradigm? Reinforcement Pre-Training

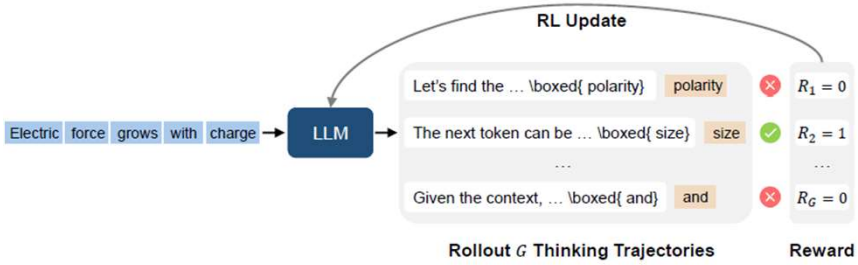
- Technical Details: Generation

지금까진 이해를 위해 reasoning 부분이
<think> ~~ </think> 로 싸여있다고 했는데 실제 이 논문
에서는 next-token을 /boxed{ } 로 둘러싸는
방식입니다.
중요한 건 아니예요



New Paradigm? Reinforcement Pre-Training

- Technical Details: Generation



$x_0\ x_1\ x_2\ x_3\ x_4\ x_5$ + Instruction

Context

Input to LLM

Pre-trained LLM

$< \text{think}^1 > \sim\sim\sim < /\text{think} > \hat{x}_6^1$

$< \text{think}^2 > \sim\sim\sim < /\text{think} > \hat{x}_6^2$

$< \text{think}^G > \sim\sim\sim < /\text{think} > \hat{x}_6^G$

Reward 'Rule'

r_6^1 r_6^2 r_6^G

Generate G responses

GRPO [2]

[2] DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, DeepSeek-AI, Apr. 2024

New Paradigm? Reinforcement Pre-Training

Technical Details: Reward Design

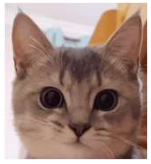
Prefix Matching Reward

To verify the correctness of y_t^i, we introduce a **prefix matching reward**, which supports verifying predictions that span multiple tokens or involve out-of-vocabulary tokens.[2] Let $\overline{x}_{\geq t}$ and $\overline{y}_t^i$ denote the byte sequences of the ground-truth completion sequence $x_{\geq t}$ and the prediction y_t^i, respectively. Denote the byte length of $\overline{y}_t^i$ by l. We define the cumulative byte lengths of the tokens in the ground-truth completion sequence as valid boundaries, and denote this set by $\mathcal{L}_{gt}$. Formally, the reward r_t^i for the i-th output for $x_{<t}$ is defined as:

$$r_t^i = \begin{cases} 1 & \text{if } \overline{y}_t^i = \overline{x}_{\geq t}[1:l] \text{ and } l \in \mathcal{L}_{gt} \\ 0 & \text{otherwise} \end{cases}, \tag{3}$$

우리가 생각했던 trivial한 reward 방식보다
조금 더 '관대하게' Pass를 주는 방식입니다.

Verifiable Reward 는 P/F 판독기가 필수!!



New Paradigm? Reinforcement Pre-Training

- Technical Details: Reward Design

Prefix Matching Reward

To verify the correctness of y_t^i, we introduce a **prefix matching reward**, which supports verifying predictions that span multiple tokens or involve out-of-vocabulary tokens.[2] Let $\overline{x}_{\geq t}$ and $\overline{y}_t^i$ denote the byte sequences of the ground-truth completion sequence $x_{\geq t}$ and the prediction y_t^i, respectively. Denote the byte length of $\overline{y}_t^i$ by l. We define the cumulative byte lengths of the tokens in the ground-truth completion sequence as valid boundaries, and denote this set by $\mathcal{L}_{gt}$. Formally, the reward r_t^i for the i-th output for $x_{<t}$ is defined as:

본 논문에서는
word를 token, token을 byte 라고 표현하고
있습니다.

New Paradigm? Reinforcement Pre-Training

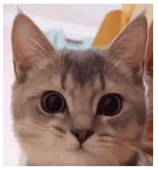
- Technical Details: Reward Design

Prefix Matching Reward

```
1  from my_tokenizers.langchain_tokenizer import LangChainTokenizer
2
3  tokenizer = LangChainTokenizer(model='hosted_vllm/deepseek-v3-0324')
4
5  print(tokenizer.encode("unhappiness", text=True))
✓ 0.6s
[Tokenizer][2025-06-23 11:55:09][DEBUG] Successfully load tokenizer for `hoste
['un', 'h', 'appiness']
```

```
1  from my_tokenizers.langchain_tokenizer import LangChainTokenizer
2
3  tokenizer = LangChainTokenizer(model='hosted_vllm/deepseek-v3-0324')
4
5  print(tokenizer.encode("happy", text=True))
✓ 0.6s
[Tokenizer][2025-06-23 11:55:14][DEBUG] Successfully load tokenizer for `host
['happy']
```

```
1  from my_tokenizers.langchain_tokenizer import LangChainTokenizer
2
3  tokenizer = LangChainTokenizer(model='hosted_vllm/deepseek-v3-0324')
4
5  print(tokenizer.encode("appy", text=True))
✓ 0.6s
[Tokenizer][2025-06-23 12:00:09][DEBUG] Successfully load tokenizer for `hoste
['appy']
```



DeepSeek–v3-0324의 tokenizer는

‘un’, ‘h’, ‘appiness’, ‘happy’, ‘appy’ 를 모두 token으로 가지고 있는 것을 알 수 있습니다.

New Paradigm? Reinforcement Pre-Training

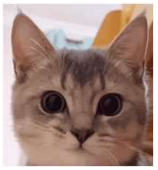
Technical Details: Reward Design

Prefix Matching Reward

```
1  from my_tokenizers.langchain_tokenizer import LangChainTokenizer
2
3  tokenizer = LangChainTokenizer(model='hosted_vllm/deepseek-v3-0324')
4
5  print(tokenizer.encode("unhappiness", text=True))
✓ 0.6s
[Tokenizer][2025-06-23 11:55:09][DEBUG] Successfully load tokenizer for `hoste
['un', 'h', 'appiness']
```

```
1  from my_tokenizers.langchain_tokenizer import LangChainTokenizer
2
3  tokenizer = LangChainTokenizer(model='hosted_vllm/deepseek-v3-0324')
4
5  print(tokenizer.encode("happy", text=True))
✓ 0.6s
[Tokenizer][2025-06-23 11:55:14][DEBUG] Successfully load tokenizer for `host
['happy']
```

```
1  from my_tokenizers.langchain_tokenizer import LangChainTokenizer
2
3  tokenizer = LangChainTokenizer(model='hosted_vllm/deepseek-v3-0324')
4
5  print(tokenizer.encode("appy", text=True))
✓ 0.6s
[Tokenizer][2025-06-23 12:00:09][DEBUG] Successfully load tokenizer for `hoste
['appy']
```



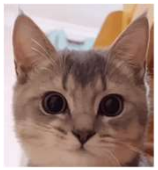
‘happy’ 는 그 자체의 token 으로 표현할 수 도 있지만,
‘h’ + ‘appy’ 의 두 token의 합으로 표현할 수도 있습니다.

New Paradigm? Reinforcement Pre-Training

- Technical Details: Reward Design

Prefix Matching Reward

If	you	say	so	,	I	am	very	happy
x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	$x_{T=8}$



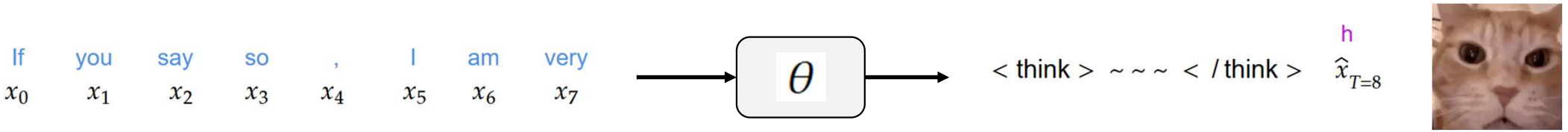
하지만 'happy' 같이 흔한 단어는
일반적으로
'happy' 단일 token으로 표현하지
'h' + 'appy' 표현하지 않습니다.

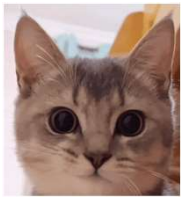
New Paradigm? Reinforcement Pre-Training

Technical Details: Reward Design

Prefix Matching Reward

If	you	say	so	,	I	am	very	happy
x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	$x_{T=8}$





Prediction Token이

1) Tokenizer에 등록된 token이고

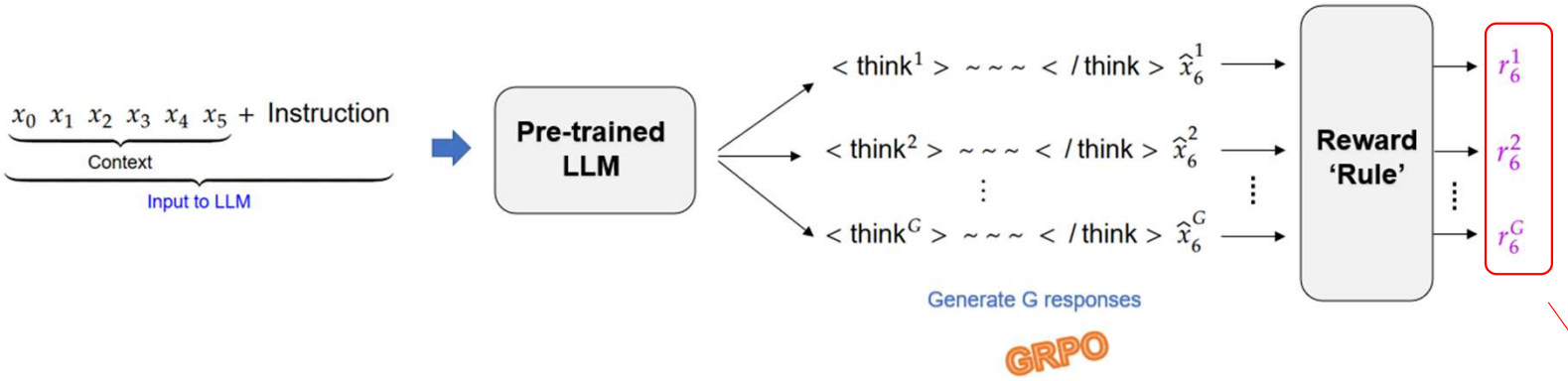
2) 정답의 'prefix'라면

맞은 것로 해주겠다는 의미입니다.

New Paradigm? Reinforcement Pre-Training

- Technical Details: Reward Design

Prefix Matching Reward



$$r_t^i = \begin{cases} 1 & \text{if } \overline{y}_t^i = \overline{x}_{\geq t}[1:l] \text{ and } l \in \mathcal{L}_{gt} \\ 0 & \text{otherwise} \end{cases},$$

Binary Reward

5. Discussion